



浙江大学

B/S 软件设计

课程设计设计报告

College of Computer Science & Technology

刘建翔 3220102363

Experiment Date: 2024-11-15

Report Hand-in Date: 2024-11-15

1. 项目概述

1.1. 项目背景

随着电商的迅速发展，消费者面对大量商品和价格信息，难以快速找到最佳价格。随着网购的普及和电商平台的不断增多，消费者常常面临相似商品在不同平台上价格差异较大的情况。消费者在购买前通常需要在多个平台间切换，以确保找到性价比最高的商品。一个集成的比价平台可以节省用户的时间和精力，提升购物体验。

1.2. 需求分析

- 用户可以使用邮箱进行注册。
- 用户可以在平台搜索任意商品，获取多个平台价格比较信息，如价格区间、价格变化、最低价等。
- 用户可以收藏某商品，以跟踪其价格变化信息，并在商品降价时收到通知
- 用户可以通过可视化的图表了解商品价格变化情况。
- 用户可自由选择在手机端或电脑端获取类似的体验。

1.3. 产品设计



比比价 平台可以对京东、淘宝等平台上的商品进行智能比价，为用户实时提供价格最低的商品信息，方便用户购物选择。用户可以使用邮箱免费注册比比价平台，并可选择在手机端、电脑端访问。产品提供以下功能：

- 在主页分类栏内选择商品进行淘宝/京东比价，也可以自行搜索商品
- 在平台上直接去往最低价格商品购买页面
- 聚类展示某一关键词商品的价格范围、平均价格等信息
- 收藏某一特定商品并接收降价通知
- 以可视化的形式呈现商品价格变化

1.4. 技术栈

1.4.1. 前端

- Vite：前端页面设计，用于构建整体页面
- vue-router：前端路由
- JS：用于前端脚本设计
- npm：前端包管理
- css：前端样式设计
- Axios：用于向后端发送请求
- Echarts：数据可视化包，用于构建图表
- Element-plus：前端组件库
- Android：手机端开发（备选项）

1.4.2. 后端

- Django : 响应请求、连接数据库
- Python : 编写后端代码
- Selenium : 用于数据爬取（备选项）
- mysql : 数据库

1.4.3. 其他

- git : 源代码管理
- docker compose : 用于运行网站的虚拟环境

2. 后端设计

本项目采取前后端分离的开发模式，后端整体开发采用 django 进行（不使用 django 提供的前端功能），并采用 mysql 作为数据库。

2.1. 数据库模型

在 django 创建模型后执行 migrate 指令即可自动在连接的数据库中创建相应的表，无需手动操作，本项目使用的几个模型如下：

具体商品模型 product_item:

字段	类型	说明
product_id	AutoIncrement	商品 id
product_name	String	商品名
query_id	ForiegnKey	搜到该商品所用的关键词 id
from	String	来自哪个平台
category_id	String	品类 id
img	Image	图片链接
code	String	条码

一个关键词会查询到多个具体商品，关键词模型 query_of_products:

字段	类型	说明
query_key	String	使用的关键词
query_id	AutoIncrement	关键词 id

商品集价格记录模型 price_time:

字段	类型	说明
----	----	----

query_id	ForeignKey	关键词 id
query_time	DateTime	更新时间
from	String	来自哪个平台
category_id	String	品类 id
min_price	Integer	关键词索引到的商品的最低价
max_price	Integer	关键词索引到的商品的最高价

用户模型 user_profile:

字段	类型	说明
user_id	AutoIncrement	用户 id
user_nickname	String	用户昵称
user_email	Email	用户邮箱
password	String	密码
avatar	Image	头像图片链接

收藏记录模型 collect_record:

字段	类型	说明
user_id	ForiegnKey	用户 id
query_id	ForiegnKey	关键词 id

2.2. 爬虫设计思路及接口

我使用 Selenium 来实现淘宝和京东网站的数据爬取。以淘宝为例，基本思路如下：

- 通过模拟点击打开淘宝的登录界面，截取登录使用的二维码返回给前端。
- 用户完成扫描后，有以下两种路径：
 - 登录后的淘宝页面维持在网站后台，这样做服务器开销较大。
 - 记录登录后产生的 cookie，当用户需要进行搜索等操作时再重新启动 Selenium 并再次载入 cookie，以恢复登录状态，实际开发中采取该方案。
- 当用户发起搜索时，再次调用淘宝页面通过模拟用户操作进行搜索，逐页爬取数据，将商品数据存入数据库，同时返回给前端展示给用户。
- 定期进行模拟点击等操作进行保活，防止淘宝登录过期。
- 若登录过期，则发送通知告知用户需要重新登录。

2.2.1. 登录爬虫 loginSpi

启动一个 Selenium 截取淘宝登录界面二维码



并将该二维码显示于网站前端，要求用户使用手机淘宝进行扫码登录，登录完成后，将登录后网页的 cookies 保存到网站数据库中，然后关闭 Selenium。

```
1  {
2      {
3          "domain": ".taobao.com",
4          "expiry": 1763034899,
5          "httpOnly": false,
6          "name": "thw",
7          "path": "/",
8          "sameSite": "Lax",
9          "secure": false,
10         "value": "cn"
11     },
12     {
13         "domain": ".taobao.com",
14         "expiry": 1731504299,
15         "httpOnly": false,
16         "name": "_m_h5_tk",
17         "path": "/",
18         "sameSite": "None",
19         "secure": true,
20         "value": "c0f0da8208f557b444fd1c9df2686a04_1731508619337"
21     },
22     {
23         "domain": ".taobao.com",
24         "expiry": 1747050900,
25         "httpOnly": false,
26         "name": "tfstk",
27         "path": "/",
28         "sameSite": "Lax",
29         "secure": false,
30         "value": "fJctirZo8eQtNQbdpt93m6nqRVtnBj3Zjcu5ioqGhDnKjkf075v4MmEK0l2g1VAjxc0W7lv2QmQZ90CeshJZlsF0G3xkZQ0N7SPXqoR7onEau2Z"
31     },
32     {
33         "domain": ".taobao.com",
34         "expiry": 1731504299,
35         "httpOnly": false,
36         "name": "mtop_partitioned_detect",
37         "path": "/",
38         "sameSite": "None",
39         "secure": true,
40         "value": "1"
41     }
42 }
```

如图所示为实际爬取的登录后 Cookies 数据，数据量较大，后期会尝试筛选出关键字段，以减少储存开销。

接收前端请求中的邮箱、密码和用户昵称，先尝试新建用户，若发生 `IntegrityError`（即邮箱重复），直接返回错误。否则使用 `auth.login` 登录刚创建的账户。后期可能添加注册时验证邮箱（向邮箱发送验证码）的功能。

2.3.3. 检验登录状态 `checkLoginState`

- 请求参数：None
- 响应参数：`isLogged`: 是否登录；`userName`: 用户昵称；`userEmail`: 用户邮箱

检查当前发送请求的 `session` 是否登录。由于请求头中已经附带了 `sessionid`，直接检查 `request.user` 是否为匿名用户即可（未登录即为匿名用户）。

2.3.4. 登出当前用户 `logout`

- 请求参数：None
- 响应参数：None

使用 `auth.logout` 登出发送请求 `session` 的用户。

2.4. 商品数据管理接口

后端商品数据按照以下逻辑进行管理：

- 网站维护一个相对固定的类目表（该表亦直接从淘宝爬取），用于用户在主页上直接索引
- 用户搜索时，将爬取的所有单个商品加入表 `product_item`。
- 提取爬取商品的最低最高价格，加入表 `query_of_products`，并更新 `price_time` 刷新此关键词商品对应的最低最高价。
- 网站定期刷新所有 `query_of_products` 中记录对应的商品。

2.4.1. 用户搜索接口 `queryProducts`

用于实现前端搜索功能。调用爬虫模块从淘宝上爬取对应关键词的商品列表，更新 `product_item`, `query_of_products`, `price_time` 等表；并返回 json 格式的商品列表

2.4.2. 获取价格变化记录 `getPriceHistory`

用于实现前端展示价格变化功能。返回 `query_id` 对应关键词价格区间变化的列表。

2.5. 用户收藏管理

用户可以收藏某一个商品关键词，并在其价格达到历史最低时接到通知。

2.5.1. 添加收藏 `addCollection(query_of_products)`

为当前 `session` 的用户添加一个关键词对应的 `query_id` 到 `collect_record` 中。（即 `(user_id, query_id)`）

2.5.2. 取消收藏 `removeCollection(query_of_products)`

删除当前用户的一条收藏记录。（即 `(user_id, query_id)`）

2.5.3. 获取用户收藏列表 `getCollectionList`

返回当前用户所有收藏的 `query_id` 列表。

2.6. 邮件发送模块

使用 `django` 提供的邮件模块，并注册一个免费的 163 邮箱账户，设置代理服务后即可利用以下方法发送邮件：

```
send_mail(  
    subject='欢迎注册比比价',
```

```
message='您的验证码为 114514',  
from_email='bibijia1011@163.com',  
recipient_list=[request.user.username],  
fail_silently=False  
)
```

3. 前端框架

前端采用 vue 搭建，所有前端页面都挂载在 App.vue 页面之上，由 vue-router 进行路由管理，用户首次进入网站将被重定向至 LoginPage.vue 进行登录或注册操作，完成后可进入主页 MainPage.vue，主页分为两栏 FrontView（搜索框页面）和 CollectionView（用户收藏页面）。使用 vue 搭建前端相当容易，由于其数据双向绑定的特性，只需要考虑具体的交互逻辑即可。网站架构使用 html+css 编写，大多数控件来源于免费的 ElementPlus 框架。

3.1. 相关配置

3.1.1. Axios

Axios 是一款易于使用的基于 promise 网络请求库，作用于前后端分离的网站中。在 main.js 中引入后，由于前后端域名不同，我们还需要解决跨域问题，由于后端使用 django，直接添加 corsheaders 插件可以简单的解决。另外，出于安全考虑，前端向后端发送请求时需要带一个 CSRF Token 凭证以防止恶意请求。具体而言需要先向后端发送一个 GET 请求获取 CSRF Token，再将其存入前端 cookies，此后向后端发送请求都需要携带该 CSRF Token。

3.1.2. Vue-Echarts

一些数据可视化控件，在 main.js 中引入即可

```
import VueECharts from 'vue-echarts'  
app.component('v-chart', VueECharts)
```

可以便捷的生成图表。

3.1.3. Vue-Router

用于静态页面跳转的路由管理，我们需要在 main.js 中创建一个路由表 routes，并建立路由，将该路由一同挂载到 app 上。

```
const router = createRouter({  
  history:createMemoryHistory(),  
  routes,  
})  
app.use(router).mount('#app')
```

网页的静态路由比较简单，目前只有 login 和 index 两个页面，之后可能还会加入新的页面。其中 index 页面有两个子路由，后面会详细解释。进入网站将会先默认重定向到 index 页。

```
const routes = [  
  {path: '/', redirect: '/index'},  
  {path: '/login', component: LoginPage},  
  {path: '/index', component: MainPage, children: [  
    {path: '', redirect: '/front'},  
    {  
      path: '/front',
```



```
        component: FrontView,  
      },  
      {  
        path: '/collect',  
        component: CollectionView,  
      },  
    ],  
  ],  
]
```

3.2. 登录页面 LoginPage.vue



Sign in

邮箱

密码

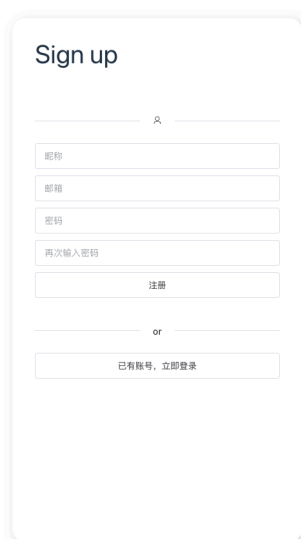
登录

or

还没账号，立即注册



每次网页被挂载时，会先发送 `checkLoginState` 请求检查是否已经登录，若没有，用户将被重定向至该页面进行登录或注册操作。登录页面中，用户在左侧表单中输入账号和密码后点击登录按钮，前端会将数据作为参数给后端发送 `login` 请求，若得到响应为 `success`，则重定向至首页 `index`，代表成功登录。



Sign up

昵称

邮箱

密码

再次输入密码

注册

or

已有账号，立即登录



用户点击“还没账号，立即注册”后，左侧栏切换为注册表单。用户填入昵称、邮箱、密码等，前端会先进行以下校验

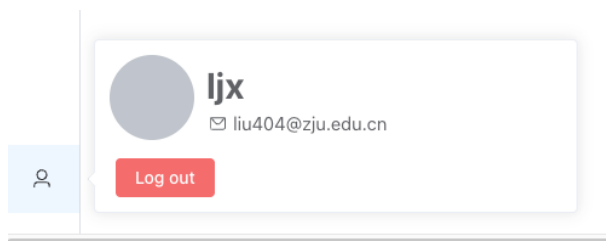
- 使用正则表达式验证邮箱是否合法
- 使用正则表达式验证密码是否安全
- 验证表单所有域非空
- 验证昵称长度
- 验证两次输入的密码一致

全部通过后才将把这些数据作为参数向后端发送 `register` 请求，由后端校验邮箱、昵称是否重复，若得到响应为 `success`，则重定向至首页 `index`，代表成功注册。用户也可以在注册页面再次点击“已有账号，立即登录”返回登录页面。

3.3. 首页 MainPage



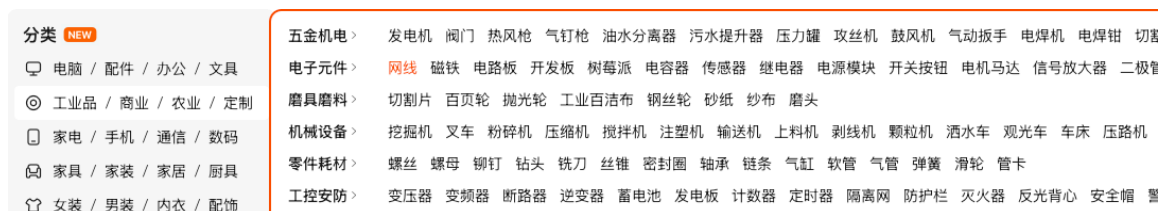
首页左侧使用 ElementPlus 提供的菜单栏，设计了三个菜单项，其中上面两个按钮“主页”、“收藏”绑定了 `index` 页面的两个子路由 `FrontView` 和 `CollectionView` (默认选中 `FrontView`) 被选中的页面将展示在右侧。从而实现菜单栏的功能。



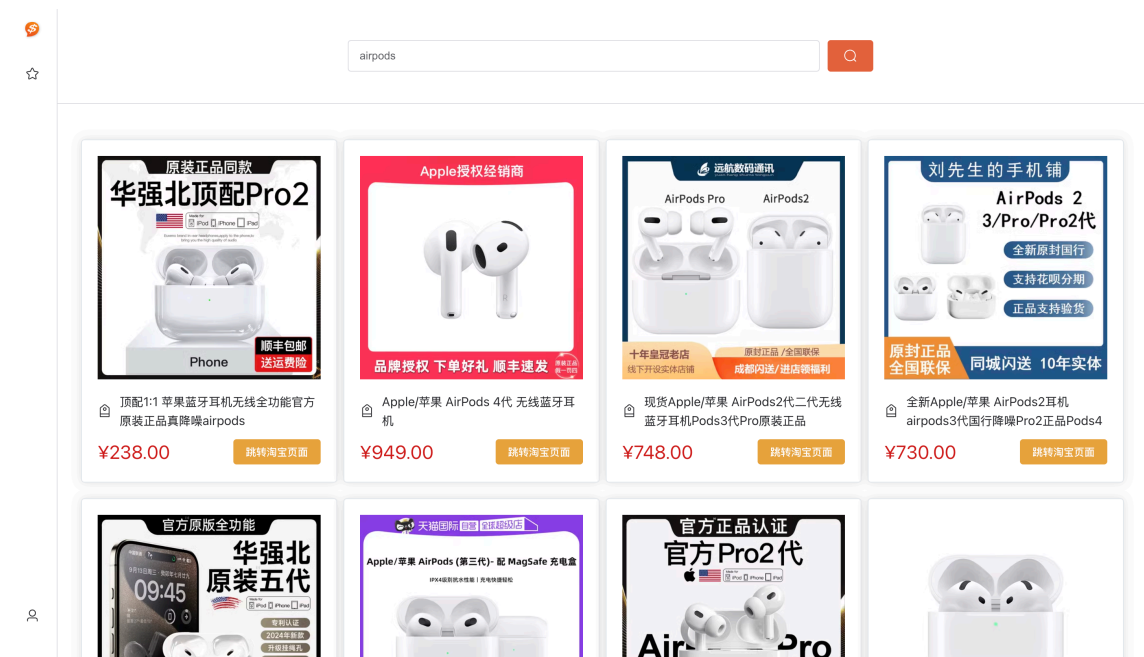
下方“用户”栏没有绑定页面路由，在按下后会打开一个 `ElPopover` 弹出框，展示当前登入用户的基本信息，并提供一些相关设置项，例如淘宝/京东登入，通知设置，登出等，之后的开发可能会设计一个页面供用户进行设置。

3.4. 主页 FrontPage

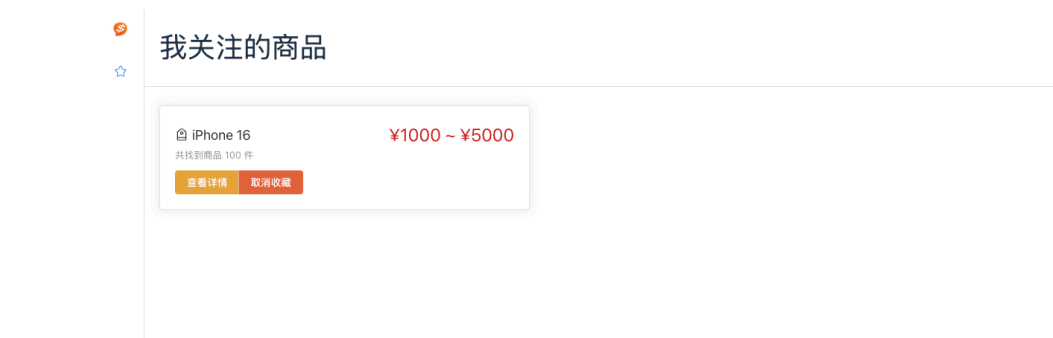
该页面上方有一个搜索框，用户可使用该搜索框进行商品检索。页面下方提供一个多层次的类目标供用户选择想看的品类，而无需输入具体关键词，类似于淘宝首页



用户输入关键词并按下搜索按钮后，页面在加载完成后会自动变为搜索结果页面，将会以卡片形式展示获取结果，若没有结果则显示空，大致效果如下：



3.5. 收藏页 CollectionView



该页面以卡片列表的形式展示当前用户收藏的所有关键词，展示价格区间。若用户想要了解有哪些商品、具体价格是多少、价格变化趋势图，可以点击“查看详情”按钮，页面将跳转到InfoPage进行展示。若不想继续关注某商品，则可直接点击取消收藏。用户可以直接点击本卡片来跳转到最低价商品在淘宝/京东的购买页面。

4. 手机端开发

由于项目采取前后端分离模式，因此开发手机端只需要重新编写前端代码即可。撰写报告时尚未开始手机端开发，目前有以下想法

- 使用 Android Studio 开发安卓应用
- 开发微信小程序（需要审核）
- 适配手机端网页

关于扫描二维码搜索商品：由于商品二维码收集较为困难（购物网站并不提供）且很多商品没有条形码，因此正在考虑该功能的必要性。即使要做，也有可能需要调用第三方 API 来实现。

5. Docker 配置

目前开发仍然在我的机器上运行，但 Docker-compose 的基本用法我已知晓。其中 Selenium 模块已经迁移至 docker 容器，并能够实现数据传输。

6. 后续开发计划

- 完善注册时验证码机制
- 完成爬虫的几个接口开发
- 完成商品信息获取/聚类/更新接口
- 完善用户收藏/降价通知功能
- 完成手机端开发
- 项目迁移至 Docker Compose